PROJETO E ANÁLISE DE ALGORITMOS — TRABALHO PRÁTICO 2

IFSULDEMINAS — Campus Poços de Caldas | Engenharia de Computação
Professor: Douglas Castilho | **Tema:** Teoria dos Grafos em Java

1. Visão Geral da Arquitetura

O projeto adota uma arquitetura orientada a objetos modularizada em dois pacotes principais, separando o armazenamento em memória da execução algorítmica:

- `src/grafos/`: Contém as interfaces base (`Grafo`, `AlgoritmosEmGrafos`), entidades (`Vertice`, `Aresta`, `TipoDeRepresentacao`, `FileManager`) e as 3 representações:
  - `GrafoMatrizAdjacencia`: Matriz booleana de adjacência e matriz de pesos $O(V^2)$.
  - `GrafoMatrizIncidencia`: Matriz de incidência vértices × arestas ($O(V \times A)$).
  - `GrafoListaAdjacencia`: Vetor de listas encadeadas ($O(V + A)$).
- `src/algoritmos/`: Classes especialistas com responsabilidade única: `BuscaEmProfundidade`, `BuscaEmLargura`, `DeteccaoCiclos`, `KosarajuCFC`, `KruskalAGM`, `DijkstraCaminhoMinimo` e `FordFulkersonFluxoMaximo`, unificadas pela classe fachada `AlgoritmosEmGrafosImpl`.
- `src/Main.java`: Menu interativo de console para testes e execução prática.
- `tests/`: Instâncias em `.txt` para piores casos (DAG acíclico, múltiplos CFCs, desconexo, rede de fluxo, grafo denso) e classe de benchmark `TestePerformance.java`.

2. O Papel dos Vetores no Projeto

O **vetor** é a base estrutural de todo o sistema:

- **Vetor de Listas de Adjacência:** O grafo é estruturado como um vetor primitivo `ArrayList<Aresta>[]`, indexado diretamente pelo ID do vértice de origem com acesso $O(1)$.
- **Matrizes Primitivas:** Arrays primitivos bidimensionais (`boolean[][]`, `double[][]`, `int[][]`) compõem as matrizes de adjacência e incidência.
- **Vetores de Rastreamento (DFS e BFS):** Vetores nativos `int[] cor`, `int[] d`, `int[] f` e `Vertice[] pi` indexados pelo ID de cada vértice.
- **Vetores em Union-Find (Kruskal):** Os conjuntos disjuntos são operados exclusivamente sobre os vetores primitivos `int[] pai` e `int[] rank`.
- **Vetores no Dijkstra:** Vetores `double[] dist`, `boolean[] visitado` e `Aresta[] arestaPai` para relaxamento e reconstrução ordenada da rota.
- **Vetores no Ford-Fulkerson:** Matriz residual `double[][] rCap` e vetor de caminho aumentante `int[] pai`.

3. Resumo dos Algoritmos Implementados

Algoritmo	Método / Conceito	Complexidade	Saída Produzida
DFS	Recursão com 3 cores (Branco, Cinza, Preto)	$O(V + A)$	Tempos d e f, antecessores π e classificação de arestas (árvore, retorno, avanço, cruzamento).
BFS	Fila FIFO concêntrica	$O(V + A)$	Distância em saltos d, antecessores π e arestas da árvore BFS.
Ciclo	Teorema da aresta de retorno na DFS	$O(V + A)$	Booleano (`true` se dígrafo tiver ciclo, `false` se for DAG).
CFC	Kosaraju-Sharir (DFS em G + DFS no transposto G^T)	$O(V + A)$	Lista de componentes e Grafo Reduzido (Condensação).
AGM	Kruskal com Union-Find em vetor	$O(A \log A)$	Arestas da árvore geradora e custo total somado.
Caminho Mínimo	Dijkstra clássico com relaxamento guloso	$O(V^2)$	Sequência ordenada de arestas de s até t e custo acumulado.
Fluxo Máximo	Ford-Fulkerson (Edmonds-Karp via BFS residual)	$O(V \cdot A^2)$	Valor numérico do fluxo máximo da rede.

4. Testes e Análise de Performance e Escalabilidade

4.1. Validação no Grafo Oficial (`docs/Teste.txt` , |V| = 5, |A| = 10)

Funcionalidade	Resultado Observado	Status
DFS (Vértice inicial 0)	4 arestas de árvore, 6 arestas de retorno (incluindo laços 0->0 e 1->1).	Aprovado
BFS (Vértice inicial 0)	Distâncias ordenadas: d[0]=0, d[1]=1, d[2]=2, d[3]=3, d[4]=4.	Aprovado
Existe Ciclo	`true` no grafo de teste; `false` em DAG de controle.	Aprovado
Componentes Fortemente Conexos	1 único componente {0, 1, 2, 3, 4}; grafo reduzido com 1 nó.	Aprovado
Árvore Geradora Mínima (AGM)	Arestas: (2-3: 1), (1-2: 4), (3-4: 7), (4-0: 32). Custo Total: **44.0**.	Aprovado
Caminho Mínimo (0 para 4)	Rota: 0 -> 1 -> 2 -> 3 -> 4. Custo Total: **62.0**.	Aprovado
Caminho Mínimo (4 para 1)	Rota: 4 -> 0 -> 1. Custo Total: **82.0** (menor que 4->3->1 que custaria 120.0).	Aprovado
Fluxo Máximo (0 para 4)	Gargalo limitante na aresta 2->3 saturando o trajeto em **1.0**.	Aprovado

4.2. Estudo Empírico de Desempenho e Limites de Viabilidade

Realizamos testes de estresse com grafos aleatórios de 50 a 5.000 vértices para determinar onde as estruturas e algoritmos atingem seus limites práticos:

Tamanho |V|	Lista Adj (Carga)	Matriz Inc (Carga)	DFS / BFS	Kosaraju	Kruskal	Ford-Fulkerson
100	0 ms	20 ms	1 ms	0 ms	13 ms	1 ms
500	0 ms	732 ms	0 ms	2 ms	29 ms	2 ms
1.000	0 ms	8.687 ms	0 ms	2 ms	80 ms	67 ms
2.500	1 ms	*Inviável (>15s)*	1 ms	6 ms	114 ms	*Lento (>5s)*
5.000	2 ms	*Inviável*	2 ms	10 ms	460 ms	*Lento (>5s)*

> **Conclusões sobre Limites de Viabilidade:**
> • **Matriz de Incidência:** Torna-se inviável a partir de **|V| ≥ 1.000** vértices devido à realocação contínua de colunas por aresta ($O(V \times A)$).
> • **Matriz de Adjacência:** Torna-se inviável por estouro de memória (`OutOfMemoryError`) a partir de **|V| ≥ 15.000** (15.000^2 posições requerem mais de 500 MB contíguos de memória heap).
> • **Ford-Fulkerson:** Atinge limite prático a partir de **|V| ≥ 2.500** em grafos densos ($O(V \cdot A^2)$).
> • **Lista de Adjacência com DFS/BFS/Kosaraju:** Mantém altíssima eficiência (≤ 10 ms) mesmo para 5.000 vértices com densidade moderada.

5. Como Compilar e Executar

Compilação de todo o projeto e testes:

```
$files = (Get-ChildItem -Path src,tests -Recurse -Filter *.java).FullName
javac -encoding UTF-8 -sourcepath "src;tests" -d bin $files
```

Execução do Menu Interativo:

```
java -cp bin Main
```

Execução dos Testes de Performance:

```
java -cp bin tests.TestePerformance
```